

LLM Evaluations: Technical Study Notes

Understanding LLM Evaluations (LLM Evals)

What is an LLM Eval?

I define **LLM Evals** as **systematic, repeatable tests used to judge an LLM or an LLM-powered system against a clear criteria**. Rather than treating it as a simple metric, you must think of an LLM Eval as the **complete testing setup** that we establish to evaluate the performance of our models and applications.

To fully understand this, we must unpack the three main characteristics that make up an LLM Eval:

1. **Systematic**: This means we are absolutely not doing "**vibe testing**". Vibe testing is when you randomly think of five questions, ask them to your chatbot, get the answers, and assume everything works fine. Instead, a systematic approach requires creating **proper datasets** that actively cover all kinds of edge cases to test the chatbot or LLM-based system thoroughly. For example, if I were building a chatbot for CampusX, I would randomly take 100 chat sessions from real users interacting with my system, compile them into a proper database or dataset, and test against that dataset to capture real-world behavior.
2. **Repeatable**: This means that if you modify your system tomorrow—whether you change the **prompt, the model, the retriever, or the chunking strategy**—you must still be able to evaluate the system in the exact same way as before. With a consistent test dataset, you can apply it across different versions of your software to extract and compare results. This allows us to objectively see whether **Version 1 (V1)** or **Version 2 (V2)** performs better and determine if the system is actually improving over time.
3. **Clear Criteria**: Proper evaluations depend heavily on the specific criteria we establish. Without a criteria, you are just doing vibe testing. If I am building the CampusX chatbot, I would define several clear criteria:
 - The generated answer must be **correct**.
 - The answer must be written using a **simple explanation**.
 - The explanation must be retrieved specifically from **our course content**.
 - The output must be **safe**—it should not contain unsafe content, abuses, or a scary tone.

Evaluating LLM Evals vs. Traditional ML Metrics

Many people transitioning from machine learning or deep learning backgrounds (including myself when I first encountered this topic) mistakenly assume that "eval" is just another word for a "metric". In classical ML/DL, evaluation is synonymous with metrics like **accuracy, precision, and recall**.

However, in the context of Large Language Models, **an LLM Eval is the entire testing setup, not just a single metric.**

This entire testing setup includes:

- **What component** we are evaluating (e.g., if we are building a RAG chatbot, the internal **retriever** component itself is part of our eval setup).
- **The criteria** used to evaluate (e.g., measuring the **retriever's accuracy**).
- **The test dataset** we prepare specifically for this evaluation.
- **The environment/timing** of the test (whether we run the evaluation **offline** during development, or **in production** after deployment).
- **The tools** being used (for example, using a tool like **Ragas** because we are evaluating a RAG-based application).

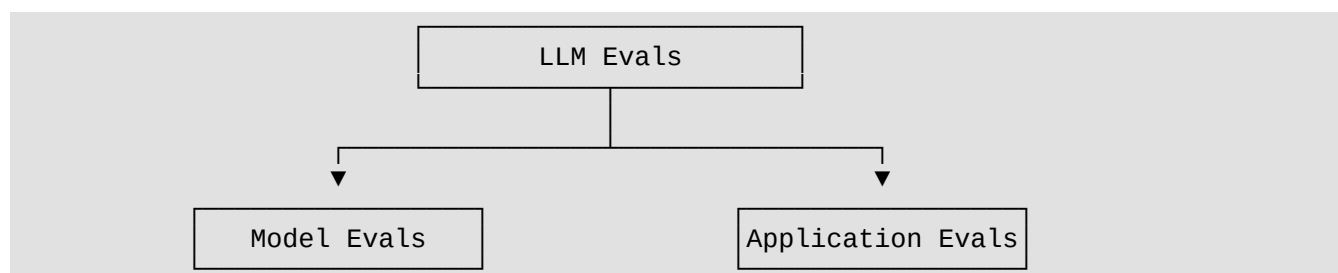
Ultimately, the goal of an LLM Eval is not to merely output a numeric score. Its purpose is to answer highly practical business and operational questions:

- Can this model be used for a particular task or application?
- Is this system good enough to ship to production?
- Did prompt version 2 actually improve over prompt V1?
- Is the RAG answer strictly grounded in the retrieved context?
- Is our agent completing its assigned task correctly?
- Is the chatbot safe for real-world users?
- Is the latency of our system under control?

The Two Main Types of LLM Evaluations

We can categorize LLM evaluations into two distinct branches: **Model Evals** and **Application Evals**.

Note: I must give you a disclaimer here. These terms—"Model Eval" and "Application Eval"—are not official industry terminology. I have created these terms specifically to simplify this complex topic for you. In the industry, both are simply referred to under the umbrella term of "LLM Evals", and people implicitly understand which one is being discussed based on the context of the conversation.



1. Model Evals

Model Evals evaluate the model itself. Their sole purpose is to test, benchmark, and document the core **capabilities of an LLM**. Whenever a new model is released, it is evaluated on public benchmarks or leaderboards, and these results are published on the internet to prove how capable the model is.

Today, we evaluate models across **eight major capability categories**:

1. **Reasoning**: Can the LLM reason through a problem and think step-by-step to solve it?
2. **Knowledge**: Does the model possess general world knowledge up to its cut-off date?
3. **Basic Maths**: Can the model successfully solve math problems?
4. **Coding**: Can the model write code?
5. **Instruction Following**: Can the model accurately follow instructions (e.g., if given 10 instructions, does it follow them one by one)?
6. **Long Context Handling**: Can the model retrieve correct answers from extremely large contexts?
7. **Multimodal Understanding**: Does the model understand and process different modalities like images, text, and sound, and generate outputs accordingly?
8. **Tool Use**: Can the model properly utilize external tools?

Famous Model Benchmarks

These capabilities are tested using standardized benchmarks:

- **MMLU**: Used to test **general knowledge and reasoning** across multiple subjects like science, history, law, and medicine.
- **GSM8K**: Used to evaluate **basic math skills** by asking grade school math questions.
- **SWE-bench & HumanEval**: Famous benchmarks used to evaluate **coding capabilities**.
- **IFEval**: Used to test **instruction following**.
- **Needle In A Haystack (NIAH)**: A famous benchmark used to evaluate **long context handling**.
- **MMMU**: Used to evaluate **multimodal capabilities**.

My Advice on Model Evals for AI Engineers

If you are pursuing a career as an AI Engineer, **you will not practically perform Model Evals on a day-to-day basis**. Evaluating, benchmarking, and documenting models is the job of massive frontier labs (like OpenAI, Anthropic, etc.) when they release a new model.

However, you still need to know what model evaluation is, understand what benchmarks exist, and know how to read them. When you start a new project, this literacy allows you to make informed decisions about whether to use an OpenAI model, an Anthropic model, or an open-source model based on which one tops the relevant benchmark.

2. Application Evals

This is the category you must study deeply. It should be your primary topic of interest and is the evaluation work you will actually perform in practice. As an AI Engineer, your primary role is to build LLM-based applications, meaning you are responsible for evaluating them.

Why Application Evals Exist

In any real-world application, **the LLM is just one single component**. While beginners often believe that LLM applications are entirely about the LLM because it acts as the "brain," experienced engineers know that a complete system requires many other elements around it to work properly.

These non-LLM components include:

- **User Interface (UI)**
- **Prompts / System Prompts**
- **Tools & APIs**
- **Orchestration Code** (such as managing branching, parallel control, and flow inside a framework like **LangGraph**)
- **Guardrails** applied at the application level
- **Output Parsers**
- **Memory & Context**
- **Retrieval System** (such as **embedding models** and **vector databases** in a RAG pipeline)
- **Monitoring Setup**
- **Feedback Loops**

The Smartphone Analogy

Think of an LLM as a mobile processor chip (like Snapdragon or MediaTek). The chip manufacturers run benchmarks and release scores (like AnTuTu) to demonstrate how strong their processor is.

But does a fast processor guarantee a great smartphone? Absolutely not. A great smartphone requires a high-quality camera system, a smooth operating system, a good sound system, a capable graphics card, solid battery life, and a great screen. All of these components must work together seamlessly. If you add a battery, you must evaluate the battery; if you add a screen or graphics card, you evaluate those components as well.

Similarly, frontier labs evaluate the "processor" (the LLM itself), but we must evaluate the entire "smartphone" (the full application system we build around that LLM).

Defining Application Evals

Application Evals assess the behavior and performance of an LLM-powered application, either at the level of the entire system or on a specific component within it.

- **System-level evaluation:** Evaluating the overall output, such as the final chat response, overall system latency, and cost per token.
- **Component-level evaluation:** Isolating and evaluating specific parts of the system, such as whether your **retriever**, **embedding model**, or **re-ranker** is working correctly.

In Application Evals, we do not ask "*Can the model do this?*" (which is a Model Eval question). Instead, we ask: "**Will our product work properly?**".

Key Questions Answered by Application Evals

Using our CampusX chatbot example, Application Evals help us answer:

- Was the student's question answered correctly?
- Was the course material utilized properly by the system?
- Was the generated answer faithful to the retrieved context?
- Was the language easy to understand for a beginner?
- Did the system hallucinate?
- Did we get the answer back quickly?
- Is our chatbot safe for real users?